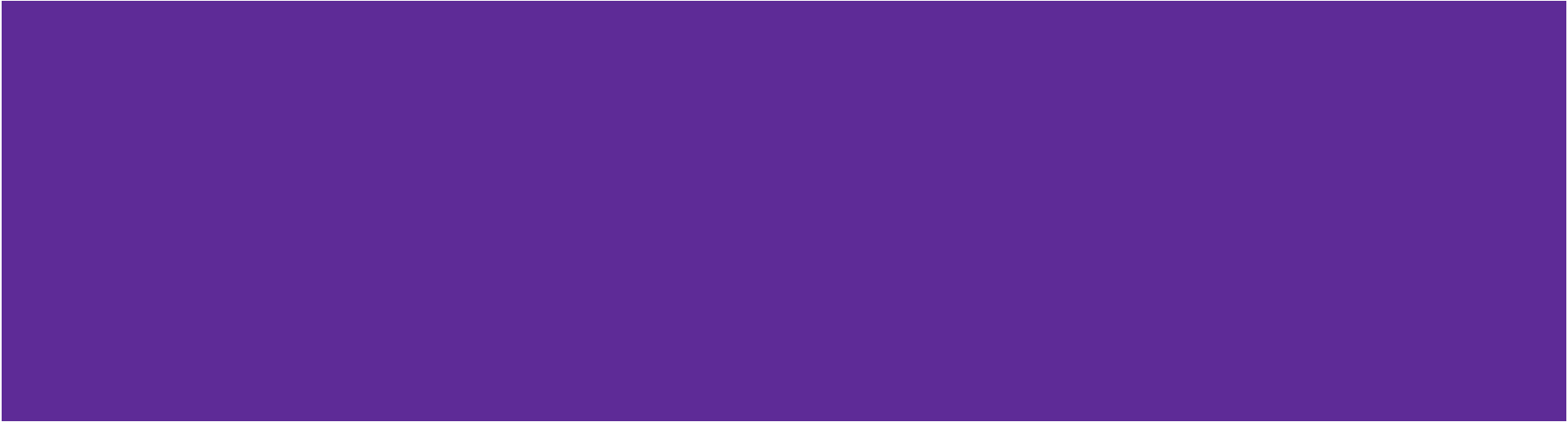


DISTRIBUTED AND CLOUD COMPUTING

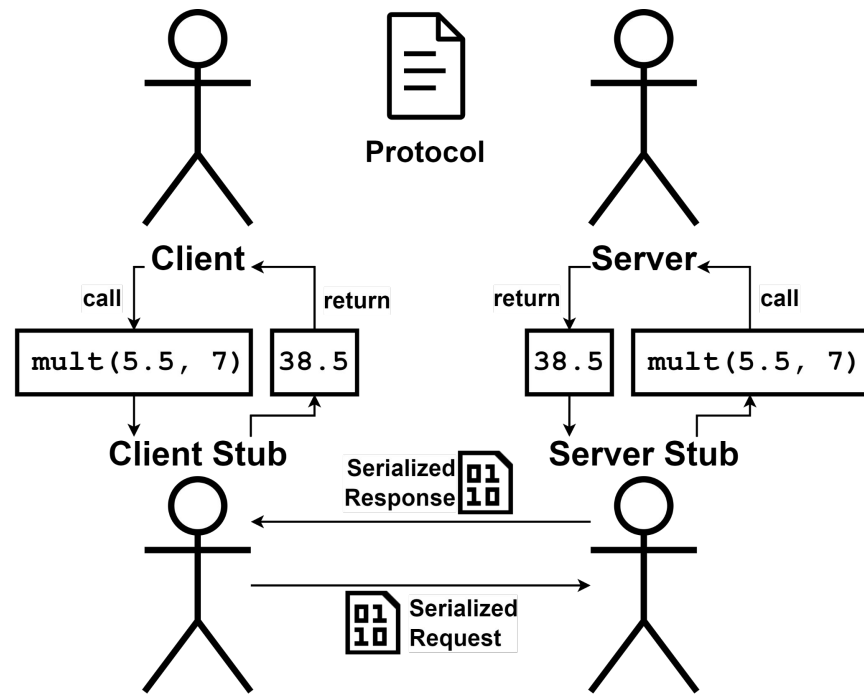
LAB 8: REVERSE PROXY + OTHER API ARCHITECTURES

(Module: Services & API Architectures)



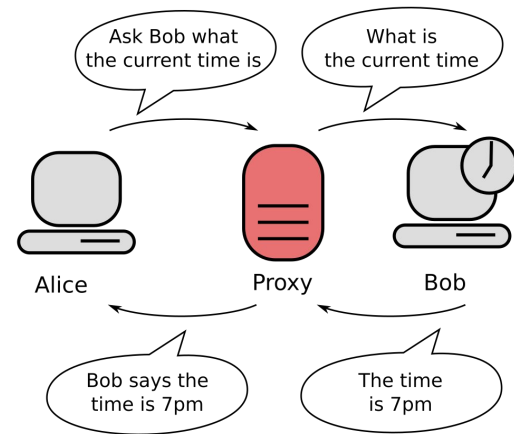
RPC Stub as A Proxy

- Client Stub
 - a. Handles serialization / deserialization of data for the client.
 - b. Handles remote server connection and data transmission for the client.
- Server Stub
 - a. Works similarly, but for the server.
- RPC Stubs act like **proxies**.
- The concept of a proxy is broad, but in computer networking, a **forward proxy** and a **reverse proxy** each have specific, distinct functions.

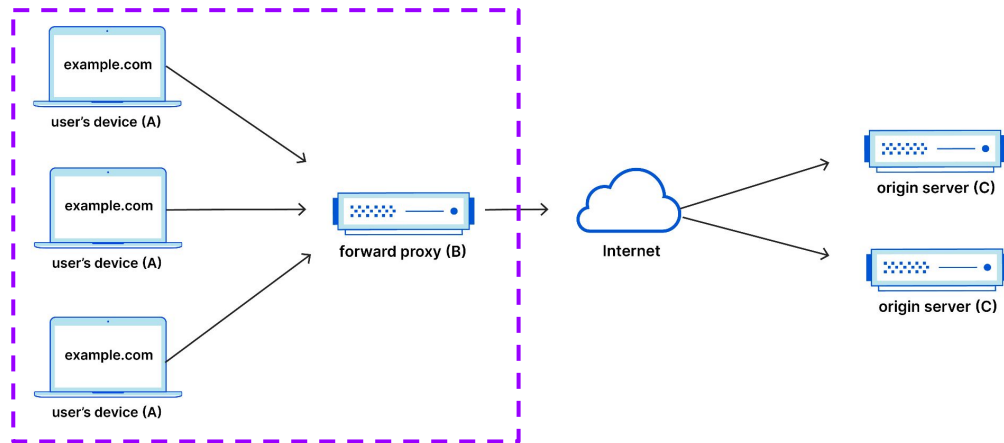


Forward Proxy

- “A forward proxy, often called a proxy, proxy server, or web proxy, is a server that sits in front of a group of client machines. When those computers make requests to sites and services on the Internet, the proxy server intercepts those requests and then communicates with web servers on behalf of those clients, like a middleman.”



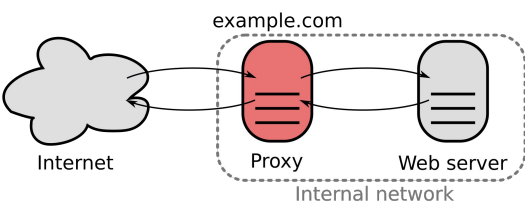
Forward Proxy Flow



Why Proxy?

1. Access Control & Security
2. Privacy & Anonymity
3. Content Caching & Filtering
4. Monitoring & Logging
5. Transparency

Reverse Proxy

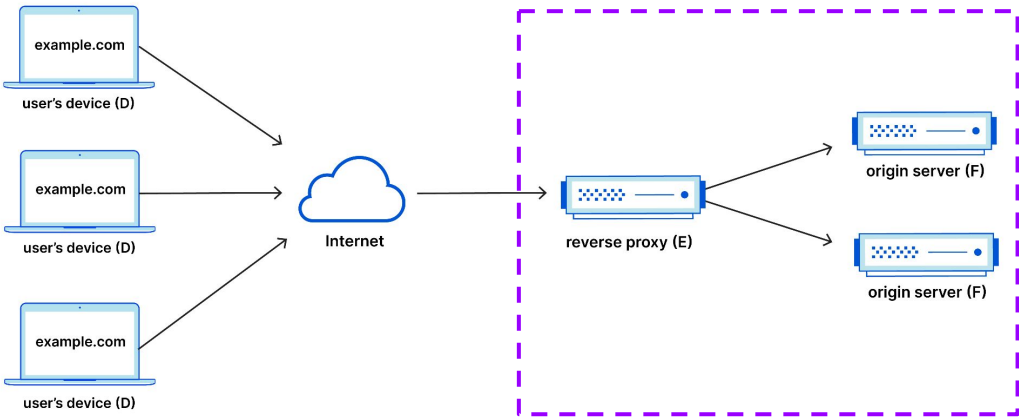


- “A reverse proxy is a server that sits in front of one or more web servers, intercepting requests from clients.”
- It works for the servers compared to the forward proxy.
- Forward & Reverse Proxy are coexisting twins.
- Example: Load Balancers, CDNs, API Gateways, Web Application Firewalls (WAFs), etc.

Why Reverse Proxy?

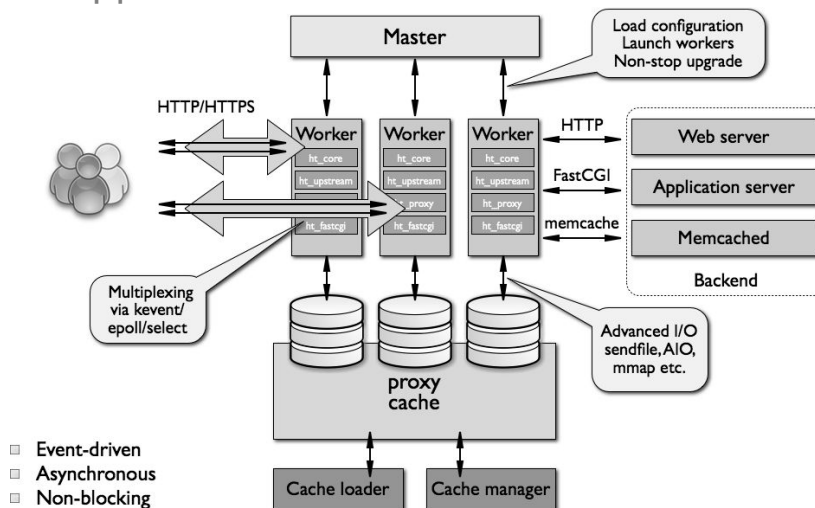
1. **Load Balancing**
2. Access Control & Security
3. Content Caching & Filtering
4. Monitoring & Logging
5. SSL Encryption Offloading

DDos attack



Nginx as A Reverse Proxy

- “nginx (“engine x”) is an HTTP web server, reverse proxy, content cache, load balancer, TCP/UDP proxy server, and mail proxy server.”
- one of the most popular tools for **load balancing** purpose
- simple configuration file format
- clean support with Docker



```

nginx.conf X
rpc_rest > 2_others > 0_reverse_proxy > nginx > nginx.conf
1 # https://nginx.org/en/docs/http/load_balancing.html
2 # placeholder: event handlers
3 events {}
4
5 # handle HTTP requests
6 http {
7     # define a group of backend servers that will handle requests
8     upstream flask_servers {
9         # Default load balancing method is round-robin.
10        # To test other methods, uncomment one of the following:
11        # least_conn; # forward to the server with the least number of
12        # ip_hash; # same ip-hash to the same server (session maintain
13        server flask_server1:5000;
14        server flask_server2:5000;
15        server flask_server3:5000;
16    }
17
18    # virtual server config
19    server {
20        listen 80;
21
22        # process requests matching the URI pattern ('/' means root + e
23        location / {
24            # forward requests to the upstream group
25            proxy_pass http://flask_servers;
26            # note back info of the original client
27            # $host: https://nginx.org/en/docs/http/nginx_http_core_module
28            # $remote_addr: https://nginx.org/en/docs/http/nginx_http_core
29            # $proxy_add_x_forwarded_for: https://nginx.org/en/docs/http/
30            # $scheme: https://nginx.org/en/docs/http/nginx_http_core_modul
31            # X-Forwarded-For records a list of hopped IPs, compared to X
32            proxy_set_header Host $host;
33            proxy_set_header X-Real-IP $remote_addr;
34            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
35            proxy_set_header X-Forwarded-Proto $scheme;
36        }
37    }
38
  
```

TASK: Reverse Proxy - Nginx

Load balance requests to 3 identical Flask servers.

> Reference codebase: [revproxy_nginx](#)

With Go [Gin](#): [revproxy_nginx_gin](#)

1. Set up Python ([uv](#) is recommended).
2. Check [flask_app/app.py](#) for the Flask server implementation. It contains 1 simple API at the base URL that returns a Hello World message marking the server's id/name.
3. Check [flask_app/Dockerfile](#) for the Flask server image specification. Build the image:
 - `docker build -t flask_app .`

TIPS: `docker image prune` to clean previous builds.

```
app.py x
rpc_rest > 2_others > 0_reverse_proxy > flask_app > app.py > ...
1  from flask import Flask
2  import os
3
4  app = Flask(__name__)
5
6  @app.route('/')
7  def hello():
8      server_name = os.getenv('SERVER_NAME', 'Unknown Server')
9      return f'Hello from {server_name}!'
10
11 if __name__ == '__main__':
12     app.run(host='0.0.0.0', port=int(os.getenv('FLASK_PORT', 5000)))
13
```

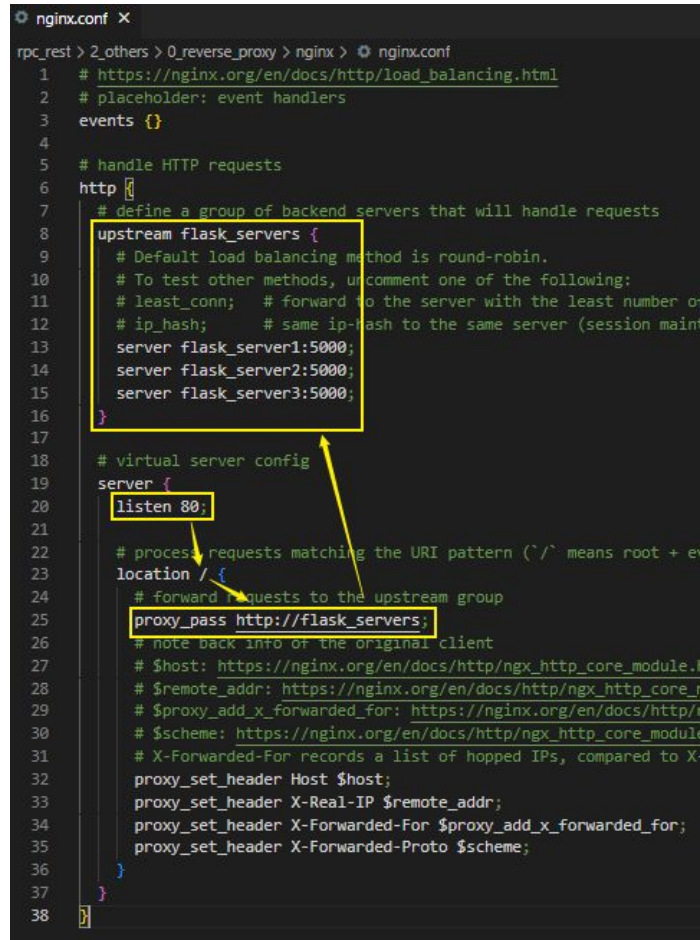
```
Dockerfile x
rpc_rest > 2_others > 0_reverse_proxy > flask_app > Dockerfile > ...
1  FROM python:3-slim
2
3  WORKDIR /app
4
5  COPY requirements.txt app.py ./
6  RUN pip3 install --no-cache-dir -r requirements.txt
7
8  CMD [ "python", "app.py" ]
9
```

TASK: Reverse Proxy - Nginx

Load balance requests to 3 identical Flask servers.

> Reference codebase: [revproxy_nginx](#)

4. Check [nginx/nginx.conf](#) that configures how nginx server intercepts & forwards the requests.
 - a. Listen for new requests on port 80.
 - b. Process requests matching / pattern only.
 - c. Forward requests to upstream server group.

A screenshot of a code editor showing the nginx.conf file. The file is titled 'nginx.conf' with a close button. The content is a configuration file for Nginx. Three specific lines are highlighted with yellow boxes: line 20 'listen 80;', line 25 'proxy_pass http://flask_servers;', and the entire 'upstream flask_servers' block from line 8 to line 16. Three yellow arrows point from the text in the list above to these highlighted sections: one from 'Listen for new requests on port 80.' to 'listen 80;', one from 'Process requests matching / pattern only.' to 'location / {', and one from 'Forward requests to upstream server group.' to 'proxy_pass http://flask_servers;'.

```
nginx.conf X
rpc_rest > 2_others > 0_reverse_proxy > nginx > nginx.conf
1 # https://nginx.org/en/docs/http/load_balancing.html
2 # placeholder: event handlers
3 events {}
4
5 # handle HTTP requests
6 http {
7     # define a group of backend servers that will handle requests
8     upstream flask_servers {
9         # Default load balancing method is round-robin.
10        # To test other methods, uncomment one of the following:
11        # least_conn; # forward to the server with the least number of
12        # ip_hash; # same ip-hash to the same server (session maint
13        server flask_server1:5000;
14        server flask_server2:5000;
15        server flask_server3:5000;
16    }
17
18    # virtual server config
19    server {
20        listen 80;
21
22        # process requests matching the URI pattern ( '/' means root + e
23        location / {
24            # forward requests to the upstream group
25            proxy_pass http://flask_servers;
26            # note back into of the original client
27            # $host: https://nginx.org/en/docs/http/nginx_http_core_module
28            # $remote_addr: https://nginx.org/en/docs/http/nginx_http_core_m
29            # $proxy_add_x_forwarded_for: https://nginx.org/en/docs/http/
30            # $scheme: https://nginx.org/en/docs/http/nginx_http_core_module
31            # X-Forwarded-For records a list of hopped IPs, compared to X
32            proxy_set_header Host $host;
33            proxy_set_header X-Real-IP $remote_addr;
34            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
35            proxy_set_header X-Forwarded-Proto $scheme;
36        }
37    }
38 }
```

TASK: Reverse Proxy - Nginx

Load balance requests to 3 identical Flask servers.

> Reference codebase: [revproxy_nginx](#)

4. Check `nginx/nginx.conf` that configures how nginx server intercepts & forwards the requests.
 - a. Listen for new requests on port 80.
 - b. Process requests matching / pattern only.
 - c. Forward requests to upstream server group.
5. Check `compose.yaml` for the defined services.
 - a. 3 Flask servers serve at port 5000, assigned with a server name.
 - b. 1 nginx server loads the `nginx.conf` file, serving at port 80 which is exposed to localhost.
6. Start the environment:
 - a. `docker compose up -d`

```
compose.yaml 1 X
rpc_rest > 2_others > 0_reverse_proxy > compose.yaml > {} services > {}
docker-compose.yml - The Compose specification establishes a standard fo
1  services:
2    # Flask servers
3    flask_server1:
4      # https://docs.docker.com/reference/compose-file
5      image: flask_app # local build
6      environment:
7        FLASK_PORT: 5000
8        SERVER_NAME: "Flask Server 1"
9
10   flask_server2:
11     image: flask_app
12     environment:
13       FLASK_PORT: 5000
14       SERVER_NAME: "Flask Server 2"
15
16   flask_server3:
17     image: flask_app
18     environment:
19       FLASK_PORT: 5000
20       SERVER_NAME: "Flask Server 3"
21
22   # Nginx for load balancing
23   nginx:
24     image: nginx:1.27
25     volumes:
26       # https://docs.docker.com/reference/compose-fi
27       - ./nginx/nginx.conf:/etc/nginx/nginx.conf:ro
28     ports:
29       - "80:80"
30     depends_on:
31       - flask_server1
32       - flask_server2
33       - flask_server3
34
```

read-only

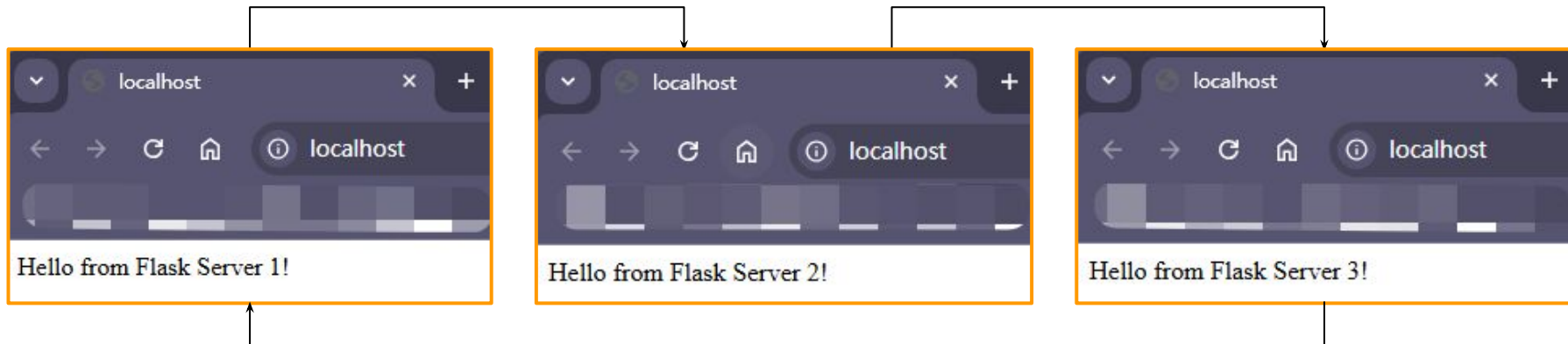
TASK: Reverse Proxy - Nginx

Load balance requests to 3 identical Flask servers.

> Reference codebase: [revproxy_nginx](#)

7. Start a web browser and access <http://localhost:80/>. Refresh the page multiple times. Note how the requests are handled by different Flask servers in a round-robin fashion (default load balancing approach of nginx).
8. Optionally, test with Postman or cURL.
 - a. e.g., `curl localhost`

```
docker compose up -d
[+] Building 1/4
✓ Network 0_reverse_proxy_default Created 0.0s
✓ Container 0_reverse_proxy-flask_server3-1 Started 0.7s
✓ Container 0_reverse_proxy-flask_server2-1 Started 0.0s
✓ Container 0_reverse_proxy-flask_server1-1 Started 0.7s
✓ Container 0_reverse_proxy-nginx-1 Started 1.0s
```



TASK: Reverse Proxy - Nginx

Load balance requests to 3 identical Flask servers.

> Reference codebase: [revproxy_nginx](#)

9. Check `test/clients.py` where 1000 requests are sent concurrently to the nginx server. Then, a map is constructed to record how many requests are handled in each Flask server. Run:

- `python test/clients.py`

```
python test/clients.py
{1: 333, 2: 333, 3: 334}
```

```
python test/clients.py
0: Hello from Flask Server 1!
1: Hello from Flask Server 3!
2: Hello from Flask Server 2!
3: Hello from Flask Server 2!
8: Hello from Flask Server 3!
5: Hello from Flask Server 1!
4: Hello from Flask Server 3!
7: Hello from Flask Server 1!
6: Hello from Flask Server 2!
9: Hello from Flask Server 1!
{1: 4, 2: 3, 3: 3}
```

10 concurrent requests

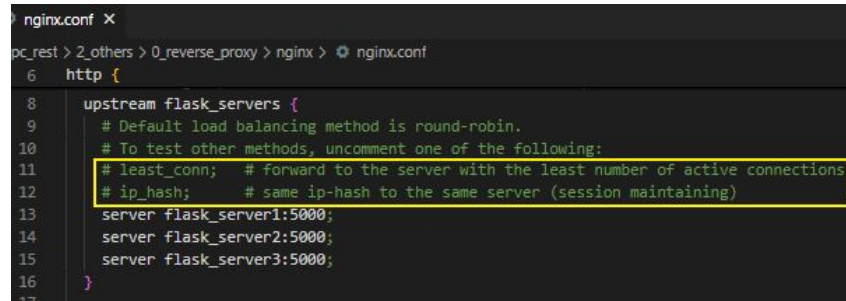
```
clients.py X
rpc_rest > 2_others > 0_reverse_proxy > test > clients.py > ...
1 import sys
2 from concurrent.futures import ThreadPoolExecutor, as_completed
3 from pprint import pprint
4
5 import requests
6
7 def send_request(id, url):
8     try:
9         response = requests.get(url)
10        msg = response.text.strip()
11        server_id = int(msg.rsplit(' ', maxsplit=1)[-1][: -1])
12    except requests.RequestException as e:
13        msg = str(e)
14        server_id = -1
15    return server_id, f'{id}: {msg}'
16
17 def test_load_balancing(url, n_requests):
18     # server_id => cnt
19     res_dict = {}
20     with ThreadPoolExecutor(max_workers=n_requests) as executor:
21         res_futures = [executor.submit(send_request, i, url) for i in range(n_requests)]
22         for res_future in as_completed(res_futures):
23             res_server_id, res_msg = res_future.result()
24             # statistics
25             if res_server_id not in res_dict:
26                 res_dict[res_server_id] = 0
27             res_dict[res_server_id] += 1
28             # sys.stdout.write(f'{res_msg}\n')
29             # sys.stdout.flush()
30     pprint(res_dict)
31
32 if __name__ == '__main__':
33     test_load_balancing(url='http://localhost:80/', n_requests=1000)
34
```

TASK: Reverse Proxy - Nginx

Load balance requests to 3 identical Flask servers.

> Reference codebase: [revproxy_nginx](#)

9. Check test/clients.py where 1000 requests are sent concurrently to the nginx server. Then, a map is constructed to record how many requests are handled in each Flask server. Run:
 - python test/clients.py
10. Switch to other load balancing methods by modifying `nginx.conf`. Restart the nginx server container manually to refresh and run the test script again.
11. Clean up the resources via:
 - `docker compose down`



```

nginx.conf X
pc_rest > 2_others > 0_reverse_proxy > nginx > ⚙ nginx.conf
6 http {
8     upstream flask_servers {
9         # Default load balancing method is round-robin.
10        # To test other methods, uncomment one of the following:
11        # least_conn; # forward to the server with the least number of active connections
12        # ip_hash; # same ip-hash to the same server (session maintaining)
13        server flask_server1:5000;
14        server flask_server2:5000;
15        server flask_server3:5000;
16    }
17

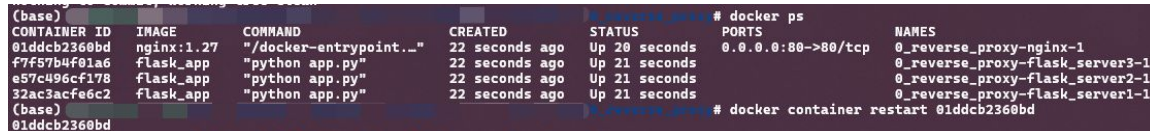
```

```
python test/clients.py
{1: 340, 2: 333, 3: 327}
```

least_conn

```
python test/clients.py
{1: 1000}
```

ip_hash



```

(base) # docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
01ddcb2360bd   nginx:1.27    "/docker-entrypoint..." 22 seconds ago Up 20 seconds 0.0.0.0:80->80/tcp                0_reverse_proxy-nginx-1
f7f57b4f01a6   flask_app     "python app.py"          22 seconds ago Up 21 seconds                               0_reverse_proxy-flask_server3-1
e57c496cf178   flask_app     "python app.py"          22 seconds ago Up 21 seconds                               0_reverse_proxy-flask_server2-1
32ac3acfe6c2   flask_app     "python app.py"          22 seconds ago Up 21 seconds                               0_reverse_proxy-flask_server1-1
(base) # docker container restart 01ddcb2360bd
01ddcb2360bd

```

SOAP

- Use XML messages + a predefined contract
- Strict, complex and verbose
- Play with a [Postman workspace](#) about SOAP.
- Check its WSDL definition by appending **?WSDL** to the service URL, or click [this link](#).

Try other services if Number service is unavailable...



```

<?xml version="1.0" encoding="utf-8"?>
<definitions xmlns="http://schemas.xmlsoap.org/wsdl/" xmlns:xs="http://www.w3.org/2001/XMLSchema" xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/" targetNamespace="http://www.dataaccess.com/webservicesserver/">
  <types>
    ...
  </types>
  <message name="NumberToWordsSoapRequest">
    <part name="parameters" element="tns:NumberToWords"/>
  </message>
  <message name="NumberToWordsSoapResponse">
    <part name="parameters" element="tns:NumberToWordsResponse"/>
  </message>
  <message name="NumberToDollarsSoapRequest">
    ...
  </message>
  <message name="NumberToDollarsSoapResponse">
    ...
  </message>
  <portType name="NumberConversionSoapType">
    ...
  </portType>
  <binding name="NumberConversionSoapBinding" type="tns:NumberConversionSoapType">
    ...
  </binding>
  <binding name="NumberConversionSoapBinding12" type="tns:NumberConversionSoapType">
    ...
  </binding>
  <service name="NumberConversion">
    ...
  </service>
</definitions>
  
```

data schema

service operation input/output message

service operation

service operation bindings to different SOAP versions

service endpoint

Public SOAP APIs / Numbers / NumberToWords

POST https://www.dataaccess.com/webservicesserver/NumberConversion.wso

Overview Params Auth Headers (9) Body Scripts Settings

raw XML

```

1 <?xml version="1.0" encoding="utf-8"?>
2 <soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
3   <soap:Body>
4     <NumberToWords xmlns="http://www.dataaccess.com/webservicesserver/">
5       <ubiNum>500</ubiNum>
6     </NumberToWords>
7   </soap:Body>
8 </soap:Envelope>
  
```

Body

```

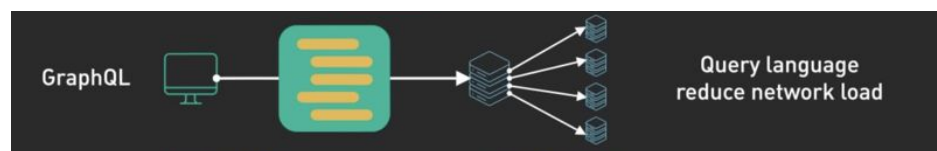
1 <?xml version="1.0" encoding="utf-8"?>
2 <soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
3   <soap:Body>
4     <m:NumberToWordsResponse xmlns:m="http://www.dataaccess.com/webservicesserver/">
5       <m:NumberToWordsResult>five hundred</m:NumberToWordsResult>
6     </m:NumberToWordsResponse>
7   </soap:Body>
8 </soap:Envelope>
  
```

200 OK - 2.60 s - 936 B

Send

GraphQL

- queries exactly needed within a single request
- suitable for complex data requirements
- Explore [Countries GraphQL API](#).
- Check schema definition [here](#).



```

80 builder.objectType(CountryRef, {
81   fields: (t) => ({
82     code: t.exposeID("code"),
83     name: t.string({
84       args: {
85         lang: t.arg.string({
86           validate: (lang) => langs(),
87         }),
88       },
89       resolve: async (country, { lang
95     },
96   })),
97   native: t.exposeString("native"),
98   phone: t.exposeString("phone").
  
```

```

123 languages: t.field({
124   type: [LanguageRef],
125   resolve: (country) =>
126     country.languages.map((code) => ({
127       ...languages[code as keyof typeof languages],
128       code,
129     })),
130 })),
  
```

```

1 query GetCountry {
2   country(code: "BR") {
3     name
4     native
5     capital
6     emoji
7     currency
8     languages {
9       code
10      name
11    }
12  }
13 }
  
```



```

{
  "data": {
    "country": {
      "name": "Brazil",
      "native": "Brasil",
      "capital": "Brasília",
      "emoji": "🇧🇷",
      "currency": "BRL",
      "languages": [
        {
          "code": "pt",
          "name": "Portuguese"
        }
      ]
    }
  }
}
  
```

+ countries.trevorblades.com

HIT 607ms

RequestID: 2382755471883397900

Variables Headers

IP limit 47 (out of 50) requests remaining (refills in 14s)



WebSocket

- enables fast, bidirectional, and persistent connections
- benefits live chat apps and real-time gaming
- Core: upgrade initial HTTP conn to WebSocket conn.
- Try [Flask-SocketIO live chat demo](#).

Flask-SocketIO Test

Async mode is: **threading**
Current transport is: **websocket**
Average ping/pong latency: **2.8ms**

Send:

hello	Echo
Message	Broadcast
Room Name	Join Room
Room Name	Leave Room
Room Name	Message
Room Name	Close Room
Disconnect	

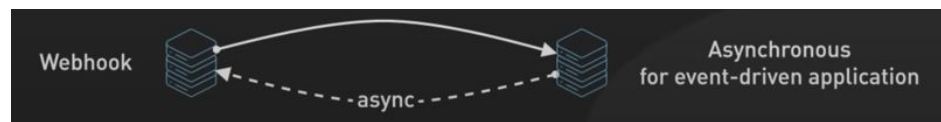
Receive:

Received #0: Connected
Received #1: I'm connected!
Received #10: Server generated event
Received #11: Server generated event
Received #2: my_event_foo[object Object]

The screenshot shows the DevTools Network tab for a WebSocket connection. The initial request is an HTTP GET to `ws://127.0.0.1:5000/socket.io/?EIO=4&transport=websocket`. The response status is 101 Switching Protocols, indicating the connection was upgraded. The Upgrade header in the request is `websocket`. Subsequent messages are received, including a 'my_event_foo' event with data `hello`.

Webhook

- supports asynchronous & event-driven notifications
- Try [Webhook.site](https://webhook.site).
- Optionally, add a new GitHub Webhook in a repository to trigger send messages upon issue-related events.
 - Create a new issue and do sth there. New messages will appear at webhook.site.
 - Remember to clean the GitHub Webhook later.



requests Actions Wiki Security Insights Settings

General

Access

Collaborators

Moderation options

Code and automation

Branches

Tags

Rules

Actions

Webhooks

Webhooks / Add webhook

We'll send a post request to the URL below with details of any sub format you'd like to receive (JSON, x-www-form-urlencoded, etc). [M documentation.](#)

Payload URL *

<https://webhook.site/be90c3aa-0079-454f-acee-3be4df1de409>

Content type *

application/json

Which events would you like to trigger this webhook?

☐ Just the push event.

☐ Send me everything.

☒ Let me select individual events.

☒ Issues

Issue opened, edited, deleted, transferred, pinned, unpinned, closed, reopened, assigned, unassigned, labeled, unlabeled, milestone, demilestoned, locked, or unlocked.

REQUESTS (0/100) Newest First

Search Query

Waiting for first request

Your unique URL

<https://webhook.site/be90c3aa-0079-454f-acee-3be4df1de409> [Open in new tab](#) [Examples](#)

access this URL to simulate an event-driven message

Your unique email address

be90c3aa-0079-454f-acee-3be4df1de409@emailhook.site [Open in mail client](#)

Your unique DNS name

[*.be90c3aa-0079-454f-acee-3be4df1de409.dnshook.site](https://be90c3aa-0079-454f-acee-3be4df1de409.dnshook.site) [About DNSHook](#)

Proxy bidirectionally with Webhook.site CLI

```
$ whcli forward --token=be90c3aa-0079-454f-acee-3be4df1de409 --target=https://localhost
```

REQUESTS (1/100) Newest First

Search Query

GET	#28811 89.213.0.57
	11/05/2024 11:15:00 PM

Request Details

GET	https://webhook.site/be90c3aa-0079-454f-acee-3be4df1de409
Host	89.213.0.57 Whois Shodan Netify Censys VirusTotal
Date	11/05/2024 11:15:00 PM (a few seconds ago)
Size	0 bytes
Time	0.000 sec
ID	28811a53-803a-45a0-ad38-c585dae45b2d
Note	Add Note

Summary

- **Reverse Proxy**
 - Reverse Proxy vs. Forward Proxy
 - Benefits:
 - i. Load Balancing
 - ii. Access Control & Security
 - iii. Content Caching & Filtering
 - iv. Monitoring & Logging
 - v. SSL Encryption Offloading
 - Nginx as a reverse proxy
- **Other API Architectures**
 - SOAP
 - GraphQL
 - WebSocket
 - Webhook

That's all for this module!

Reverse Proxy Flow

